

Object-Oriented Programming in Java

Lecture 8 - Exception Handling

Emily Lucia Antosch

HAW Hamburg

03.10.2025

Contents

1. Introduction	2
2. Exception Handling	6
3. Throwing exceptions	12
4. Catching exceptions	17
5. Defining custom exceptions	33
6. License Notice	41

1. Introduction

- In the last lecture, we dealt with creating graphical user interfaces
- You can now
 - ▶ create windows in which other elements can live,
 - ▶ arrange elements using layouts and panels
 - ▶ and create graphics directly in Java.
- Today we continue with **Exception Handling**.

1.1 Where are we now?

1. Imperative Konzepte
2. Klassen und Objekte
3. Klassenbibliothek
4. Vererbung
5. Schnittstellen
6. Graphische Oberflächen
7. **Exception Handling**
8. Input and Output
9. Multithreading (Parallel Computing)

1.2 The goal of this chapter

- You handle exceptions and errors that occur during program execution to establish an orderly program flow in exceptional situations.
- You define your own exception classes adapted to the needs of your specific application.

2. Exception Handling

2.1 Introductory Example

2. Exception Handling

? Question

- What output is produced by the following program?

```
1 public class ProvokeException {
2     public static void main(String[] args) {
3         int a = 3;
4         int b = 2;
5         printRatio(a, b);
6         System.out.println("Exiting main()");
7     }
8
9     public static void printRatio(int a, int b) {
10        int ratio = a / b;
11        System.out.println("Ratio = " + ratio);
12    }
13 }
```

 Java

? Question

- And what output is produced for a = 7 and b = 0?

? Question

- What can go “wrong” in a program?
- When must the normal program flow be interrupted?
- When must a program be terminated, when can it be continued?



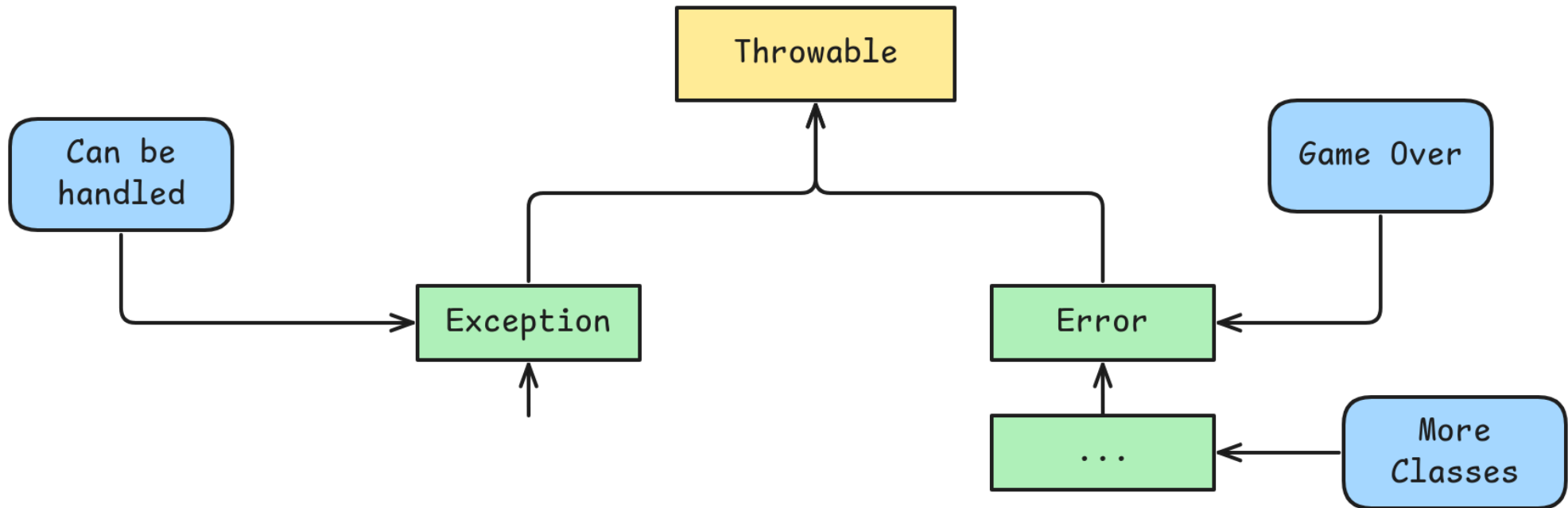
Example

- Division by zero
- Calling `a.method()` although variable `a` has the value `null`
- Negative or too high index for arrays
- Converting the string “This is text” to an integer of type `int`
- File not found
- No more memory available

2.1 Introductory Example

2. Exception Handling

- Exceptions and errors are represented by objects of special classes
- Base class of all exception classes is Throwable

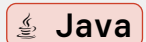


- One distinguishes:
 - ▶ Exception: Handleable, program can be continued
 - ▶ Error or fatal error: Not handleable, terminate program

! Memorize

- Exception is also used as a collective term for exceptions and errors.
 - Exception handling is also called Exception handling.
-
- Some classes for exceptions:
 - ▶ Division by zero (ArithmeticException)
 - ▶ Access to method or attribute via null reference (NullPointerException)
 - ▶ Invalid array index (ArrayIndexOutOfBoundsException)
 - ▶ Invalid characters when reading a number (NumberFormatException)
 - ▶ File not found (FileNotFoundException)

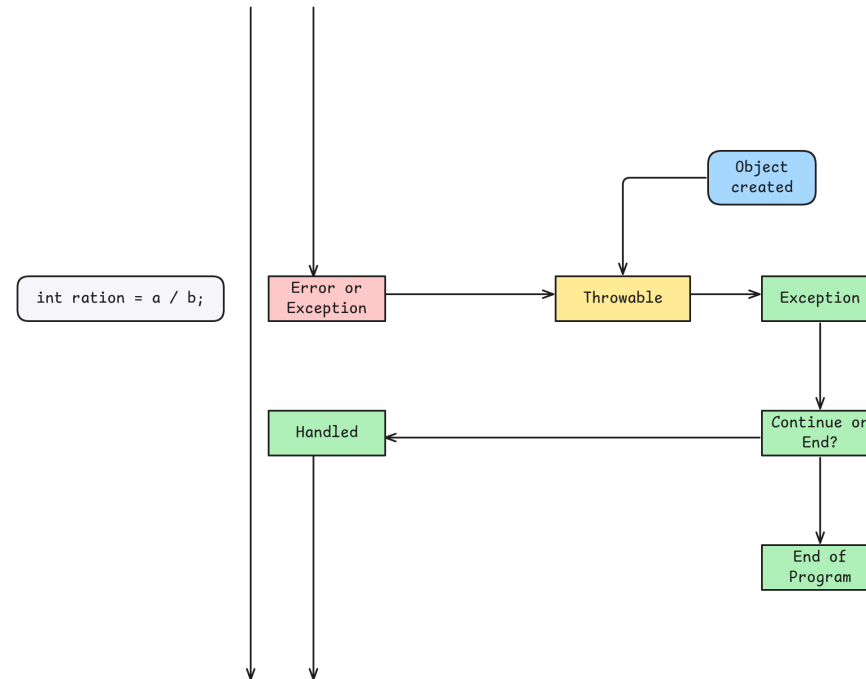
```
1 int[] array = {1, 2, 3, 4};
2 System.out.println(array[4]);
3
4 String message;
5 System.out.println(message.length());
6
7 int code = Integer.parseInt("12a4");
```



2.2 Exception handling flow

2. Exception Handling

1. Throwing an exception:
 - Program flow is immediately interrupted
 - Object is created that represents the exception
2. Catching an exception:
 - Programmer can catch and handle the exception



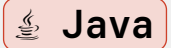
3. Throwing exceptions

3.1 Throwing exceptions

3. Throwing exceptions

- In case of error, exceptions are automatically generated (e.g. division by zero).
- However, exceptions can also be thrown explicitly.

```
1 throw ExceptionObject;
```

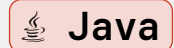


- Constructor can be passed a string (e.g. as error description)



Example

```
1 throw new Exception();  
2 throw new Exception("Division by zero");  
3 Exception exception = new Exception(); throw exception;
```

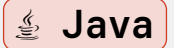


3.1 Throwing exceptions

3. Throwing exceptions

- For illustration:
 - ▶ Throw an exception before attempting to divide by zero.

```
1  public class ThrowException {
2      public static void main(String[] args) {
3          int a = 3;
4          int b = 0;
5          printRatio(a, b);
6          System.out.println("Exiting main()");
7      }
8
9      public static void printRatio(int a, int b) {
10         int ratio = a / b;
11         System.out.println("Ratio = " + ratio);
12     }
13 }
```

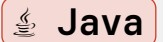


3.1 Throwing exceptions

3. Throwing exceptions

- Example solution:

```
1  public class ThrowException {
2      public static void main(String[] args) {
3          int a = 3;
4          int b = 0;
5          printRatio(a, b);
6          System.out.println("Exiting main()");
7      }
8
9      public static void printRatio(int a, int b) {
10         if (b == 0) {
11             throw new ArithmeticException("Division by zero");
12         }
13         System.out.println("Ratio = " + (a / b));
14     }
15 }
```



Java

3.1 Throwing exceptions

3. Throwing exceptions

- Output in case of error:
 - ▶ Exception type (e.g. `ArithmeticException`)
 - ▶ Error message (e.g. "Division by zero")
 - ▶ Stack trace (i.e. chain of called methods)



Example

```
1 Exception in thread "main" java.lang.ArithmeticException: Division by zero at
2     kapitel8_exceptions.ThrowException.printRatio(E02_ThrowException.java:20)
   at
3     kapitel8_exceptions.ThrowException.main(E02_ThrowException.java:14)
```

- Method `main()` called `printRatio()` at line 14
- Method `printRatio()` threw the exception at line 20

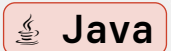
4. Catching exceptions

4.1 Exception Handling

4. Catching exceptions

- Exceptions can be caught and handled:

```
1  try {  
2      // Statements ...  
3  } catch (ExceptionType e) {  
4      // Statements ...  
5  }
```



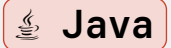
- Try block contains code that can throw an exception
- If an exception is thrown in the try block:
 1. Try block is immediately terminated
 2. Catch block is executed if the exception type (ExceptionType) matches
 3. Program continues after the catch block
- Exception type of catch block doesn't match: Exception is not caught!
- No exception thrown: Catch block is skipped

4.1 Exception Handling

4. Catching exceptions

- Avoid the “crash”:
 - ▶ Catch the thrown exception!

```
1  public class TryCatch {
2      public static void main(String[] args) {
3          int a = 3;
4          int b = 0;
5          printRatio(a, b);
6          System.out.println("Exiting main()");
7      }
8
9      public static void printRatio(int a, int b) {
10         int ratio = a / b;
11         System.out.println("Ratio = " + ratio);
12     }
13 }
```

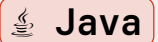


4.1 Exception Handling

4. Catching exceptions

- Example solution:

```
1  public static void printRatio(int a, int b) {
2      try {
3          int ratio = a / b;
4          System.out.println("Ratio = " + ratio);
5      } catch (ArithmeticException e) {
6          System.out.println("Exception caught in printRatio()");
7          System.out.println("e.getMessage(): " + e.getMessage());
8          System.out.println("e.toString(): " + e + "\n");
9      }
10     System.out.println("Exiting printRatio()");
11 }
```



- Selected methods for exception objects:
 - ▶ getMessage()
 - ▶ printStackTrace()
 - ▶ toString()

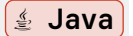
4.1 Exception Handling

4. Catching exceptions

? Question

And now?

```
1 public class TryCatchChain1 {
2     public static void main(String[] args) {
3         int ratio = getRatio(3, 0);
4         System.out.println("Ratio = " + ratio);
5         System.out.println("Exiting main()");
6     }
7     public static int getRatio(int a, int b) {
8         int ratio = 0;
9         try {
10             ratio = a / b;
11         } catch (ArithmeticException e) {
12             System.out.println("Exception caught in getRatio()");
13         }
14         System.out.println("Exiting getRatio()");
15         return ratio;
16     }
17 }
```



Java

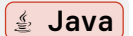
4.1 Exception Handling

4. Catching exceptions

? Question

And now?

```
1 public class TryCatchChain2 {
2     public static void main(String[] args) {
3         try {
4             int ratio = getRatio(3, 0);
5             System.out.println("Ratio = " + ratio);
6         } catch (ArithmeticException e) {
7             System.out.println("Exception caught in main()");
8         }
9         System.out.println("Exiting main()");
10    }
11
12    public static int getRatio(int a, int b) {
13        int ratio = a / b;
14        System.out.println("Exiting getRatio()");
15        return ratio;
16    }
17 }
```

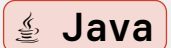


4.1 Exception Handling

4. Catching exceptions

- If multiple exception types can occur, multiple catch blocks are needed.
- Exception types of catch blocks must be different
- The first matching catch block is executed.

```
1  try {  
2      // ...  
3  } catch (ExceptionType1 e) {  
4      // ...  
5  } catch (ExceptionType2 e) {  
6      // ...  
7  } catch (ExceptionType3 e) {  
8      // ...  
9  }
```



Java

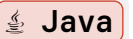
4.1 Exception Handling

4. Catching exceptions

? Question

- The following source code contains two error sources. Which ones?
- What output does the program produce?

```
1  public class ExceptionTypes1 {
2      static int recursiveIncrease(int i) {
3          return recursiveIncrease(i + 1);
4      }
5
6      public static void main(String[] args) {
7          int[] a = new int[4];
8          try {
9              a[4] = recursiveIncrease(7);
10         } catch (ArrayIndexOutOfBoundsException e) {
11             System.out.println("Caught ArrayIndexOutOfBoundsException");
12         }
13         System.out.println("Exiting main()");
14     }
15 }
```



4.1 Exception Handling

4. Catching exceptions

Task 1

- Modify the previous source code so that both error sources are caught.

```
1 public class ExceptionTypes2 {
2     static int recursiveIncrease(int i) {
3         return recursiveIncrease(i + 1);
4     }
5
6     public static void main(String[] args) {
7         int[] a = new int[4];
8         try {
9             a[4] = recursiveIncrease(7);
10        } catch (ArrayIndexOutOfBoundsException e) {
11            System.out.println("Caught ArrayIndexOutOfBoundsException");
12        } catch (StackOverflowError e) {
13            System.out.println("Caught StackOverflowError");
14        }
15        System.out.println("Exiting main()");
16    }
17 }
```

Java

4.1 Exception Handling

4. Catching exceptions

? Question

- Oops, something goes wrong in the catch block below!
- Is the new exception handled? What is output?

```
1 public class ExceptionTypes3 {
2     static int recursiveIncrease(int i) {
3         return recursiveIncrease(i + 1);
4     }
5     public static void main(String[] args) {
6         int[] a = new int[4];
7         try {
8             a[4] = 0;
9         } catch (ArrayIndexOutOfBoundsException e) {
10             recursiveIncrease(7);
11         } catch (StackOverflowError e) {
12             System.out.println("Caught StackOverflowError");
13         }
14         System.out.println("Exiting main()");
15     }
16 }
```

 Java

4.1 Exception Handling

4. Catching exceptions

- A catch block only refers to its associated try block.
- If catch block throws an exception, it is not caught by subsequent blocks

? Question

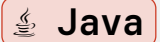
- How can we catch the exception generated in the catch block?

4.1 Exception Handling

4. Catching exceptions

- Source code that generates exception in nested try block

```
1  public static void main(String[] args) {
2      int[] a = new int[4];
3      try {
4          a[4] = 0;
5      } catch (ArrayIndexOutOfBoundsException e1) {
6          try {
7              recursiveIncrease(7);
8          } catch (StackOverflowError e2) {
9              System.out.println("Caught inner StackOverflowError");
10         }
11     } catch (StackOverflowError e) {
12         System.out.println("Caught outer StackOverflowError");
13     }
14     System.out.println("Exiting main()");
15 }
```



4.1 Exception Handling

4. Catching exceptions

- Sometimes certain code must be executed in any case.
- Example: Closing open files or data streams
- Optional finally block:
 - ▶ Always comes last (i.e. after try and catch blocks)
 - ▶ Code is executed at the end of the construct ... really always ... honestly!

```
1  try {  
2      // ...  
3  } catch (ExceptionType1 e) {  
4      // ...  
5  } catch (ExceptionType2 e) {  
6      // ...  
7  } finally {  
8      // Is guaranteed to be executed  
9  }
```



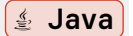
4.1 Exception Handling

4. Catching exceptions

? Question

- What is output?

```
1 public class TryCatchFinally1 {
2     static int recursiveIncrease(int i) {
3         return recursiveIncrease(i + 1);
4     }
5     public static void main(String[] args) {
6         int[] a = new int[4];
7         try {
8             a[4] = 0;
9         } catch (ArrayIndexOutOfBoundsException e1) {
10             recursiveIncrease(7);
11             System.out.println("Caught ArrayIndexOutOfBoundsException");
12         } finally {
13             System.out.println("Finally");
14         }
15         System.out.println("Exiting main()");
16     }
17 }
```



4.1 Exception Handling

4. Catching exceptions

? Question

- What is output?

```
1 public class TryCatchFinally2 {
2     public static void main(String[] args) {
3         System.out.println("Ratio = " + getRatio(3, 0));
4     }
5     public static int getRatio(int a, int b) {
6         int ratio = 0;
7         try {
8             ratio = a / b;
9         } catch (ArithmeticException e) {
10             System.out.println("Exception caught in getRatio()");
11             return 0;
12         } finally {
13             System.out.println("Finally");
14         }
15         System.out.println("Exiting getRatio()");
16         return ratio;
17     } }
```

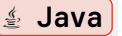


4.1 Exception Handling

4. Catching exceptions

- Rules for blocks:
 - ▶ Exactly one try block as the first block
 - ▶ None or any number of catch blocks after the try block
 - ▶ None or one finally block as the last block
 - ▶ A try block must have at least one catch or finally block.
- The following structure is allowed:

```
1 try {  
2     // ...  
3 } finally {  
4     // ...  
5 }
```



5. Defining custom exceptions

5.1 Custom exceptions

5. Defining custom exceptions

- Let's consider the following program:

```
1  public class OwnException1 {
2      public static void main(String[] args) {
3          double x = 25.0;
4          System.out.printf("sqrt(%f) = %f", x, squareRoot(x));
5      }
6
7      public static double squareRoot(double x) {
8          return Math.sqrt(x);
9      }
10 }
```



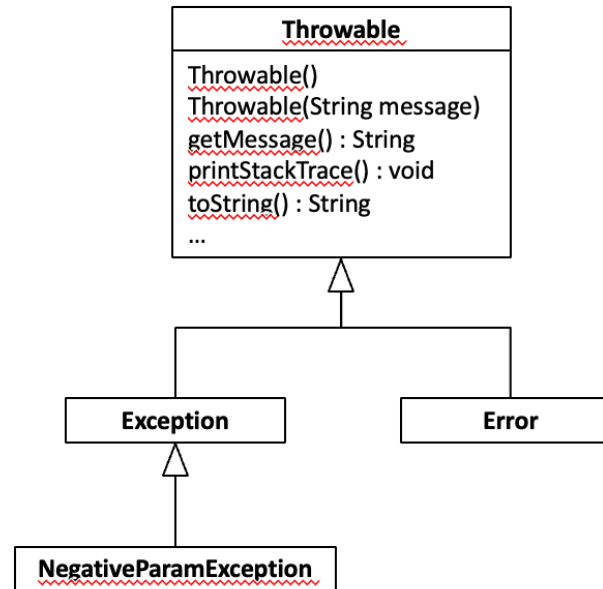
? Question

- Method `squareRoot()` should throw an exception for negative parameters
- How could we define our own type (e.g. `NegativeParameterException`)?

5.1 Custom exceptions

5. Defining custom exceptions

- Custom exception type by deriving from an existing class
- First approach: Derive from the Exception class



5.1 Custom exceptions

5. Defining custom exceptions

- Approach generates error message (“Unhandled exception”)
- Why is that now?!

```
1  class NegativeParamException extends Exception {
2  }
3
4  public class OwnException2 {
5      public static void main(String[] args) {
6          double x = 25.0;
7          System.out.printf("sqrt(%f) = %f", x, squareRoot(x));
8      }
9
10     public static double squareRoot(double x) {
11         if (x < 0.0) {
12             throw new NegativeParamException();
13         }
14         return Math.sqrt(x);
15     }
16 }
```



Java

5.1 Custom exceptions

5. Defining custom exceptions

- Background:
 - ▶ Exceptions must be caught OR
 - ▶ Method must declare via throws that it can throw an exception type.

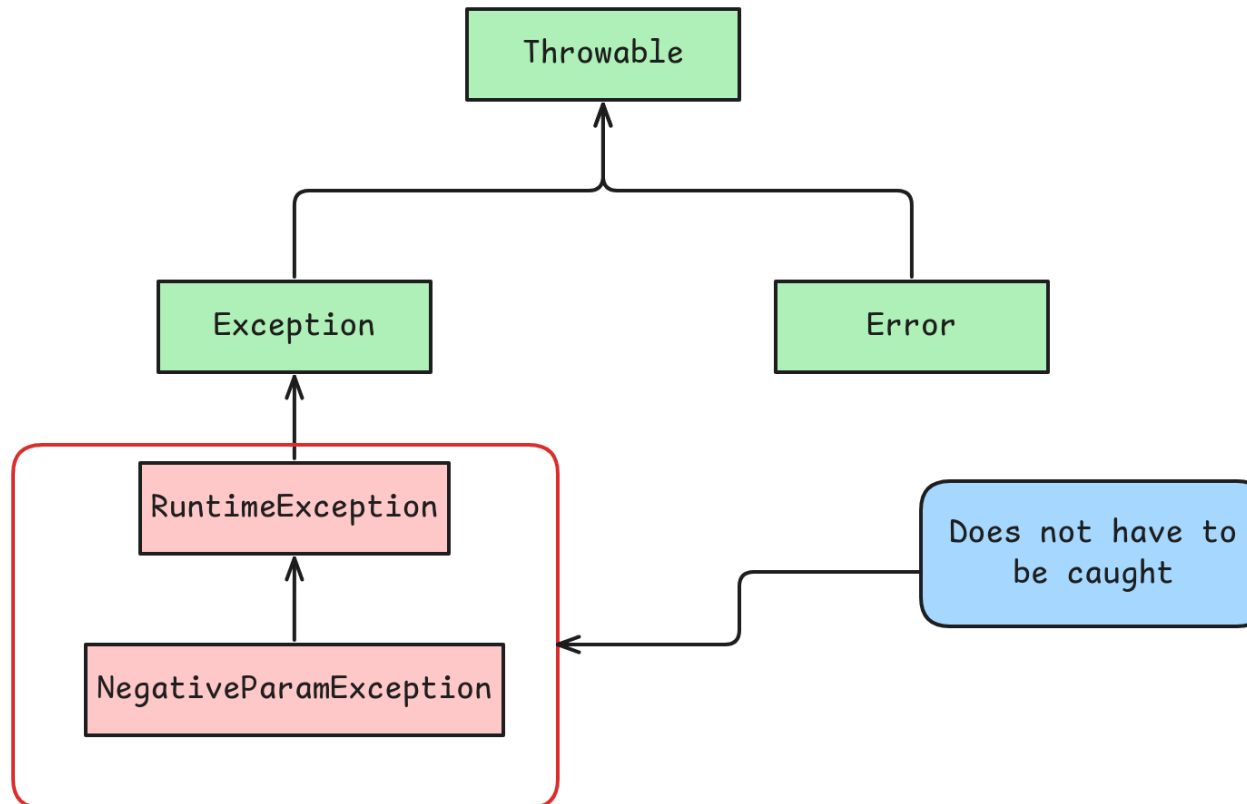
```
1  public class OwnException2 {
2      public static void main(String[] args) throws NegativeParamException {
3          double x = 25.0;
4          System.out.printf("sqrt(%f) = %f", x, squareRoot(x));
5      }
6
7      public static double squareRoot(double x) throws NegativeParamException {
8          if (x < 0.0) {
9              throw new NegativeParamException();
10         }
11         return Math.sqrt(x);
12     }
13 }
```



5.1 Custom exceptions

5. Defining custom exceptions

- This applies to all exception types (i.e. Throwable and derived from it) except for:
 - ▶ Class RuntimeException
 - ▶ Classes derived (directly or indirectly) from RuntimeException



5.1 Custom exceptions

5. Defining custom exceptions

```
1  class NegativeParamException extends RuntimeException {
2  }
3
4  public class OwnRuntimeException {
5      public static void main(String[] args) {
6          double x = 25.0;
7          System.out.printf("sqrt(%f) = %f", x, squareRoot(x));
8      }
9
10     public static double squareRoot(double x) {
11         if (x < 0.0) {
12             throw new NegativeParamException();
13         }
14         return Math.sqrt(x);
15     }
16 }
```

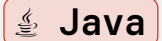


5.1 Custom exceptions

5. Defining custom exceptions

- Description (“message”) passed to constructor of base class

```
1  class MyException extends Exception {
2      public MyException(String message) {
3          super(message);
4      }
5  }
6
7  public class OwnExceptionWithMessage {
8      public static void main(String[] args) {
9          try {
10             throw new MyException("An exception just for fun :-) ...");
11         } catch (MyException e) {
12             System.out.println("Message: " + e.getMessage());
13         }
14     }
15 }
```



6. License Notice

6.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- `link("https://creativecommons.org/licenses/by-nc-sa/4.0/")`
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.